maximum depth, the minimum number of points per node, and the split criterion. With a support-vector machine, you could tune the regularization term and the kernel. Since the performance of a model very much depends on its hyperparameters, hyperparameter tuning is an important component of any ML pipeline.

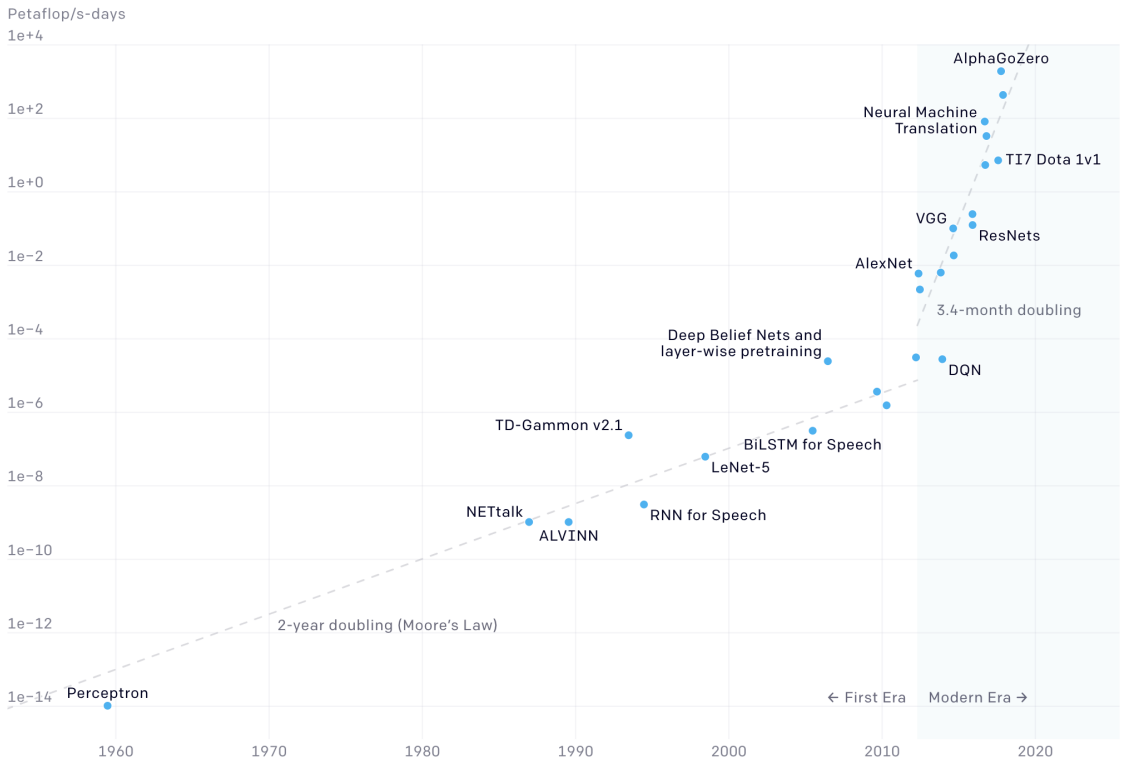


Figure 1.1 Compute cost of training large neural networks has been steadily growing, making hyperparameter tuning increasingly difficult.

If this is a typical real-world data set, this process could take a lot of time and resources. Figure 1.1 from OpenAI (https://openai.com/blog/ai-and-compute/) shows that as neural networks keep getting larger and deeper, the amount of computation necessary (measured in petaflop/s-days) increases *exponentially*.

This is to say that training a model on a large data set is quite involved and takes significant effort. Further, you want to identify the hyperparameter values that give the best accuracy, so training will have to be done many times. How should you go about choosing which values to parameterize your model so that you can zero in on the best combination as quickly as possible? That is the central question in hyperparameter tuning.

Getting back to our neural net example in Table 1.1, what is the number of layers that we should try next, so that we could find a higher accuracy than 81%? Some value between 10 layers and 20 layers is promising, since at both 10 and 20, we do better than having 5 layers. But what exact value we should inspect next is still not obvious, since there may still be a lot of variability

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>